Learning Adaptive Multi-Timescale Scheduling for Mobile Edge Computing

Yijun Hao, Shusen Yang, *Senior Member, IEEE*, Fang Li, Yifan Zhang, Shibo Wang, and Xuebin Ren, *Member, IEEE*

***Abstract*—In mobile edge computing (MEC), resource scheduling is crucial to task requests' performance and service providers' cost, involving multi-layer heterogeneous scheduling decisions. Existing MEC schedulers typically adopt static-timescale scheduling, where scheduling decisions are updated regularly at fixed intervals for all layers. The inflexible updating timescales lead to poor performance in the production networks. In this paper, we propose EdgeTimer, an unprecedented approach that automatically and adaptively determines respective updating timescales of multiple scheduling layers to achieve a better trade-off between the operation cost and service performance. Specifically, we design (i) a three-layer hierarchical deep reinforcement learning (DRL) framework for efficient learning of tightly coupled policies, (ii) a tailored multi-agent DRL algorithm for decentralized scheduling, with the convergence strictly proved, and (iii) a lightweight system defender for deterministic reliability assurance. Furthermore, we apply EdgeTimer to a wide range of Kubernetes scheduling rules, and evaluate it using production traces with different workload patterns. Through extensive trace-driven experiments, we demonstrate that EdgeTimer can significantly decrease the operation cost for service providers without sacrificing the delay performance, thereby improving overall profits, compared with the state-of-the-art approaches.**

***Index Terms*—Mobile edge computing, resource scheduling, adaptive timescales, reinforcement learning.**

I. Introduction

EFFICIENTLY utilizing edge resources—which are often more limited compared to cloud resources—matters for optimizing service performance, operation costs, and reliability in mobile edge computing (MEC) [1], [2]. Resource scheduling is a critical technique to achieve this. It works by providing a set of scheduling decisions to assign available resources to task requests *properly* [3]. For better utilization, the resources of both cloud and edge servers should be viewed as a unified whole that can be jointly scheduled, despite their inherent differences in capacity and duty [4]. Thus, efficient scheduling decisions exploit the collaboration of cloud servers, neighbor under-utilized edge servers, and the local edge server. Typically, a three-layer heterogeneous architecture is presented by segregating the overall scheduling process into layers of scheduling decisions performed in different collaboration manners, including *Layer-1: edge-cloud scheduling, Layer-2: edge-edge scheduling, and Layer-3: intra-edge scheduling* [5].

The three layers of scheduling decisions are not executed independently. Instead, they *work together* to fulfill task requests. The decision-making process across different layers follows a specific sequence and exhibits correlations. For example, edge-cloud service placement [6], edge-edge task offloading [7] and intra-edge resource allocation [8] are three representative cases corresponding to the above three layers, respectively. These decisions are made sequentially to determine which cloud services to pre-place in which edge servers before task arrivals, which servers to offload arrived tasks, and how many resources to allocate to offloaded tasks, respectively.

The majority of current MEC schedulers [4], [9], [10], [11] utilize a *static single-timescale* mode for updating scheduling decisions. In this mode, the decisions are updated at a fixed interval that is *uniform* across all three layers, as shown in Fig. 1(a). Typically, the updating interval is very short, such as one second, to ensure timely adjustments based on real-time requests. However, frequent decision adjustments result in *high operation costs* for service providers, such as expenses associated with service migration [12], handover [13], and resource reconfiguration [14].

Only a few MEC schedulers [6], [15] adopt different updating intervals across multiple scheduling layers, but these intervals are still fixed, which we refer to as the *static multi-timescale* mode (see Fig. 1(b)). Typically, they initially determine the updating timescale for the lowest intra-edge scheduling layer. Then, a *constant integer* scaling ratio is used to determine the differences in timescales between the lowest layer and other layers. As a result, when higher-layer decisions are updated, cascading updates occur in all layers below. Therefore, these schedulers cannot adapt to environment variations. Furthermore, service providers have to manually iterate through multiple cycles to identify an appropriate scaling ratio that allows them to save costs while ensuring high-performance services. This process is time-consuming and incurs additional labor costs.

Received 23 February 2024; revised 14 December 2024; accepted 2 March 2025. Date of publication 5 March 2025; date of current version 3 July 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2021YFB2401300, Grant 2022YFA1004100, and Grant 2020YFA0713900 and in part by the National Natural Science Foundation of China under Grant 61772410, Grant 61802298, Grant 62172329, Grant U1811461, Grant U21A6005, and Grant 11690011. Recommended for acceptance by A. Abdrabou. *(Corresponding author: Shusen Yang.)*

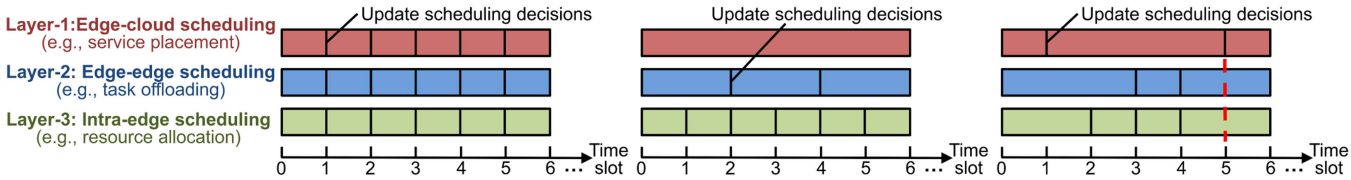
Yijun Hao, Fang Li, Yifan Zhang, Shibo Wang, and Xuebin Ren are with the National Engineering Laboratory for Big Data Analytics (NEL-BDA), Xi'an Jiaotong University, Shaanxi 710049, China.

Shusen Yang is with the National Engineering Laboratory for Big Data Analytics (NEL-BDA), Ministry of Education Key Lab for Intelligent Networks and Network Security (MOE KLINNS Lab), Xi'an Jiaotong University, Shaanxi 710049, China (e-mail: shusenyang@mail.xjtu.edu.cn).

Digital Object Identifier 10.1109/TMC.2025.3548533

1536-1233 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on June 30,2026 at 07:30:06 UTC from IEEE Xplore. Restrictions apply.



(a) Static single-timescale scheduling. (b) Static multi-timescale scheduling. (c) Adaptive multi-timescale scheduling.

Fig. 1. Timescales of different resource scheduling frameworks in MEC. The red dotted line shows the asynchrony nature of the adaptive multi-timescale framework, i.e., EdgeTimer, where the higher-layer scheduling decision is updated but lower-layer decisions remain unchanged.

This naturally poses a crucial question: *How to automatically and adaptively determine respective timescales of multiple layers of scheduling decisions to achieve a better trade-off between the operation cost and service performance?* To solve this problem, we propose EdgeTimer, an *adaptive multi-timescale* scheduling framework for edge-cloud, edge-edge, and intra-edge scheduling, as shown in Fig. 1(c). EdgeTimer has the following three desired features:

F1: Adaptation to environment variations. The updating timescales for each scheduling layer are *time-varying*, adapting to the changing patterns of online task requests.

F2: Asynchrony over multi-layer scheduling. The updating timescales for each layer are determined *independently*, without being strongly linked by a scaling ratio. This feature decouples inter-layer dependencies and achieves a more flexible updating mechanism. As a result, higher-layer decisions can be updated while lower-layer decisions remain unchanged, as illustrated by the dotted red line in Fig. 1(c).

F3: Autonomy for edge servers. Traditional centralized scheduling leads to significant communication costs due to frequent information exchanges between central cloud servers and edge servers [15]. To reduce communication costs, each edge server is expected to determine its timescales *locally*, without resorting to the computation on remote clouds.

To realize the promise, one straightforward approach is to use machine learning, especially deep reinforcement learning (DRL) [16], to learn update policies, i.e., deciding whether to update current scheduling decisions of each layer for each edge server at each slot. However, DRL is not a panacea. When applying off-the-shelf DRL to our problem, there are three major challenges to address.

The first challenge lies in the fact that conventional DRL algorithms are designed for high-level aggregated goals. In our problem, the overall goal is defined as service providers' profits that quantify the cost-performance trade-off. Achieving this goal relies on learning three layers of tightly coupled policies simultaneously. This makes the learning task complicated and intractable due to inter-layer dependencies. To address this, we propose a three-layer hierarchical DRL framework. It decomposes the learning task into independent sub-tasks with individual sub-goals, which improves learning efficiency by mitigating the impact of improper policies.

For the second challenge, conventional DRL algorithms usually use one agent for centralized decision-making, and are incapable of decentralized scheduling tasks due to non-stationary environment issues [17]. To this end, for each layer of DRL controllers, we design a multi-agent DRL algorithm to learn layer-specific decentralized policies. To tackle the lack of global information, the non-local information from other agents is used for ease of offline training, while it is not observed during the online inference process.

Third, DRL algorithms face safety issues during the online inference process due to the inherent uncertainty of deep neural networks. To address this concern, we categorize unsafe decisions in our problem, and propose a system defender that incorporates a set of safety constraints to identify each type of unsafe decision. Then, we develop tailored masking rules to repair unsafe decisions before applying them to the network system. This approach ensures a deterministic guarantee for system reliability. Furthermore, we conduct theoretical analysis to evaluate the convergence and computational complexity of EdgeTimer with the above domain-specific techniques.

To evaluate EdgeTimer's effectiveness, we establish a practical profit model that allows service providers to adjust the weight between operation costs and service performance using profit weight factors. Then, we implement a realistic Vienna system-level simulator [18] and integrate 45 typical scheduling rules, including Kubernetes rules [19] and other widely-used rules, as built-in scheduling rules. Our evaluation is conducted on multi-pattern workload traces derived from Alibaba's production clusters [20]. By applying EdgeTimer to varying profit weight factors, the results show that EdgeTimer improves the service providers' profit by up to $9.1\times$ over existing schedulers under the same rule. Notably, this improvement is achieved without sacrificing the scheduling delay performance.

In summary, our contributions can be stated as follows:

- We propose EdgeTimer, the first work to automatically and collaboratively optimize the updating timescales of multi-layer scheduling decisions to improve the service providers' profits.
- We design a three-layer hierarchical DRL algorithm to learn a hierarchy of coupled policies with low complexity, a multi-agent DRL algorithm for decentralized scheduling, and a system defender for reliability guarantee.
- Theoretical analysis and extensive experiments demonstrate the adaptability, convergence, efficiency and generalization of EdgeTimer compared with existing scheduling approaches.
- Service providers can use EdgeTimer as a plug-and-play approach. EdgeTimer is *not intrusive* to current scheduling approaches of service providers. It only provides an update signal when it is necessary. Service providers can decide

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on June 30,2026 at 07:30:06 UTC from IEEE Xplore. Restrictions apply.

whether to rely on the signal based on their extensive experience. Alternatively, providers can keep using their well-tested tools to generate scheduling decisions. Besides the application case in this paper, they can extend EdgeTimer to other single or multiple layers of scheduling decisions by simply changing the state and reward settings of DRL controllers.

Summary of changes: This paper is an extended and enhanced version of our previous conference paper [21]. Compared with the conference version, this journal version includes significant improvements in the following five aspects: (i) We have added theoretical analysis of convergence and time complexity of EdgeTimer (*Section IV-D*). (ii) We have established a more realistic profit model, allowing service providers to customize the impact of service performance and operation costs on their overall profits (*Section III-D*). (iii) To provide a deterministic reliability guarantee, we have systematically proposed a system defender component that comprehensively addresses all unsafe actions in our problem (*Section IV-C*). (iv) We have added extensive experiments about generalization (*Section VI-B*), scalability (*Section VI-E*), and in-depth analysis of key parameters (*Section VI-D*). (v) We have further discussed applicable scenarios and limitations of our work (*Section VII*).

II. Related Work

Profit optimization in MEC: Existing practices on profit optimization in MEC are mainly achieved by reducing operation costs [12], [14], [22], [23], [24], [25], [26] or optimizing pricing strategies [27], [28], [29], [30], [31], [32], [33], [34], [35]. Many works are developed on reducing operation costs by modifying scheduling policies for service placement [22], [23], [24], [25], task offloading [12], and resource allocation [14], [26]. Another line of works aim to optimizing the price of resources based on uniform pricing [27], [28], [29], [30], differentiated pricing [34], [35], and auction-based pricing strategies [33], to benefit profits. EdgeTimer opens a new dimension for profit maximization by optimizing and coordinating the update frequency of decisions in multi-layer scheduling. Furthermore, we validate the effectiveness of EdgeTimer using a profit model with varying weights assigned to operation costs and service quality.

Static-timescale scheduling in MEC: Existing MEC schedulers utilize either static single-timescale or static multi-timescale modes for multi-layer heterogeneous scheduling. In static single-timescale scheduling [4], [9], [10], [11], scheduling decisions are updated uniformly at a fixed interval across all scheduling layers. Static multi-timescale scheduling is often applied to scheduling problems that contain multiple sequential decision-making for stability and cost saving. Existing works adopt two timescales to update decisions for service placement and task offloading [6], [15], and task offloading and resource allocation [36]. However, updating timescales in all the above works are fixed over time, synchronous at a fixed interval, and consistent for all edge servers. EdgeTimer first provides an adaptive, asynchronous, and autonomous multi-timescale scheduling framework in MEC.

DRL for scheduling in MEC: DRL has been widely used in MEC to provide adaptive scheduling decisions for service placement [15], task offloading [37], [38], [39], resource allocation [40], [41], and server reservation [10]. EdgeTimer integrates three tailored designs with vanilla DRL to generate adaptive timescales for multi-layer heterogeneous scheduling decisions. First, it adopts a three-layer hierarchical DRL framework to learn coupled policies of edge-cloud, edge-edge, and intra-edge scheduling for low complexity. Second, we use a multi-agent learning algorithm in EdgeTimer for decentralized scheduling. Third, EdgeTimer incorporates safe learning scheme to ensure system reliability and accelerate learning.

TABLE I
List of Key Notations

Notations	Descriptions
$\mathcal{T}$	The set of time slots. The slot index is t
$\mathcal{N}$	The set of edge servers. The server index is i
$\mathcal{S}$	The set of services. The service index is s
o	The cloud server
$x_{i,s}(t)$	The binary indicating if service s is placed at edge i at slot t
$y_{i,j,s}(t)$	The binary indicating if a task of service s is offloaded from edge i to edge j at slot t
$z_{i,s}(t)$	The amount of computation resources allocated by edge i to service task s at slot t
$g_i(t)$	The revenue obtained by server i at slot t
$(c_i^1(t), c_i^2(t), c_i^3(t))$	$c_i^1(t)$, $c_i^2(t)$, and $c_i^3(t)$ are operation costs for service placement, task offloading, and resource allocation of edge i at slot t, respectively
$(a_i^1(t),a_i^2(t),a_i^3(t))$	$a_i^1(t)$, $a_i^2(t)$, and $a_i^3(t)$ indicate if current placement, offloading, and allocation decisions at edge i need to be updated, respectively

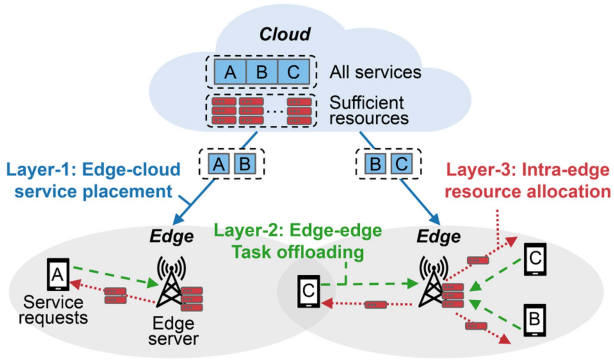


Fig. 2. The illustration of three-layer heterogeneous scheduling in MEC.

III. System Model and Problem Formulation

Our objective is to optimize the overall profit by updating scheduling decisions with an *adaptive* timescale in heterogeneous MEC systems. For clarity, Table I lists all key notations.

A. MEC System Model

As shown in Fig. 2, we consider an MEC system proceeding in discrete time slots $\mathcal{T} = \{0, 1, \ldots, t, \ldots\}$. N edge servers, $\mathcal{N} = \{1, \ldots, N\}$, are located at respective base stations, each of which is equipped with limited computation, storage, and communication resources. All edge servers are connected via back-haul links [6]. A cloud o with sufficient resources is connected to edge servers by core networks [4]. The cloud caches all

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on June 30,2026 at 07:30:06 UTC from IEEE Xplore. Restrictions apply.

services $\mathcal{S} = \{1, \ldots, S\}$. These servers are typically managed by different service providers. For simplicity, we use a single provider as an example to demonstrate how EdgeTimer maximizes its profit. Furthermore, EdgeTimer can be easily extended to jointly optimize the overall profit of multiple providers, as discussed in Section VII.

B. Resource Scheduling Model

At each time slot, task requests arrive at edge servers stochastically. Efficient processing of tasks relies on three-layer heterogeneous scheduling with different collaborations, including *Layer-1: edge-cloud scheduling, Layer-2: edge-edge scheduling, and Layer-3: intra-edge scheduling* [5]. We apply EdgeTimer to three typical cases corresponding to the above three layers, i.e., edge-cloud service placement [6], edge-edge task offloading [7], and intra-edge resource allocation [8], as shown in Fig. 2. Please note that our work is not limited to these cases. It can be easily applied to any MEC scheduling cases, such as intra-edge admission control [42], by modifying the definitions of DRL components (details in Section VII).

Layer-1: edge-cloud service placement: To process task requests from applications, e.g., machine learning inference, servers must pre-cache application-specific services, e.g., trained models and related databases [11]. The cloud caches all services with sufficient storage space. Considering the storage limits of edge servers, only a subset of services can be placed at each edge server. Services can be migrated from the cloud to edge servers, which requires efficient edge-cloud collaboration. Edge-cloud service placement provides the decision $x_{i,s}(t)$ to determine whether service s is placed at edge server i at slot t.

Layer-2: edge-edge task offloading: All edge servers with the required services are able to process task requests, but their service quality varies. Some edge servers have long queues and scarce resources, while others have plenty of idle resources. Task offloading utilizes the edge-edge collaboration. Specifically, tasks arrived at the local edge server can be offloaded to an under-utilized neighbor edge server that has cached corresponding services for better resource utilization [4]. The decision of task offloading $y_{i,j,s}(t)$ determines whether a task of service s is offloaded from edge server i to server j at slot t. The offloading process consumes the communication resources between the original and destination servers, and causes extra transmission delay.[1]

Layer-3: intra-edge resource allocation: After offloading tasks to destination edge servers, there may be several tasks competing for resources at the same edge server. The intra-edge resource allocation typically studies how to efficiently allocate a server's limited computation, communication, and storage resources to multiple task requests [8]. In this paper, we focus on the efficient allocation of computation resources. We denote the decision of resource allocation as $z_{i,s}(t)$ which indicates how much computation resources are allocated by edge server i to offloaded service task s at slot t.

[1]If there is no eligible edge server, EdgeTimer will automatically accelerate the update of service placement decisions. Here, tasks are not allowed to be outsourced to the remote cloud to avoid high transmission delay.

C. Operation Cost Model

Too frequent adjustment of scheduling decisions, i.e., $x_{i,s}(t)$, $y_{i,j,s}(t)$, and $z_{i,s}(t)$, would increase operation costs.

Specifically, placing a service s requires the edge server i to download non-trivial data from server j of a neighbor edge or a remote cloud, which incurs the operation cost, $C^1_{i,j,s}$, defined by the service type and transmission distance [6]. Following a general assumption [6], removing an installed service does not increase the cost. The cost of placing a new service s at edge server i is $C^1_{i,s} = \min_{j \in \{j | x_{j,s}(t-1)=1\}} C^1_{i,j,s}$. The operation cost for service placement of edge i at slot t can be expressed as:

$$c^1_i(t) = \sum_{s \in \mathcal{S}} C^1_{i,s} \mathbb{I}_{\{1\}}(x_{i,s}(t) - x_{i,s}(t-1)), \tag{1}$$

where $\mathbb{I}_{\{1\}}(x)$ is an indicator function that has value of 1 if $x \in \{1\}$ and 0 otherwise. Given a service s, if $\mathbb{I}_{\{1\}}(x_{i,s}(t) - x_{i,s}(t-1)) = 1$, it means that this service is newly placed at server i at time slot t and was not cached there before.

Offloading a task s from the original destination server k to the current destination server m incurs the operation cost $C^2_{k,m,s}$, including the data transmission cost and service interruption cost caused by the inter-server handover [13]. Please note that we use the indices k and m instead of i and j because, in the later cost definition, three distinct server indices are required: two for the offloading destinations of tasks from the same source server in two consecutive time slots, and one for the source server of these tasks. For clarity, we use i to denote the source server of tasks, while k and m are used to represent their destination servers in two consecutive time slots. The offloading cost of edge i at slot t is defined as:

$$c^2_i(t) = \sum_{s \in \mathcal{S}} C^2_{k,m,s} \mathbb{I}_{\{1\}}(y_{i,k,s}(t-1) y_{i,m,s}(t)), \tag{2}$$

where the indicator function equals 1 only if, at slot $t-1$, task s was offloaded from server i to server k, and at time slot t, task s is offloaded from server i to server m. In this way, (2) calculates the offloading cost of server i at slot t, resulting from changes in the offloaded destination server.

Changing resource allocation decisions may lead to resource reconfiguration, and incurs the resource lead time caused by the server booting and initialization [14], [26]. Since shutting down a server is much faster than booting it, we use the increase in resources allocated as the operation cost of resource reconfiguration. Let C^3_i be the one-unit reconfiguration cost of the computation resource. At time slot t, the allocation cost of edge server i can be represented as:

$$c^3_i(t) = \sum_{s \in \mathcal{S}} C^3_i (z_{i,s}(t) - z_{i,s}(t-1))^+, \tag{3}$$

where the function $(x)^+ = \max(\mathrm{x}, 0)$.

D. Weighted Profit Model

The profit earned by service providers can be defined by subtracting their operation cost from their revenue [43]. Service providers get revenue by selling computation resources to

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on June 30,2026 at 07:30:06 UTC from IEEE Xplore. Restrictions apply.

users [44]. Various factors influence the unit price of computation resources. For example, the unit price usually decreases as the billing cycle increases in real markets [45]. It is difficult to incorporate all influencing factors into a unified and precise pricing function. Alternatively, we observe that influence factors like the billing cycles vary over time. This means that the unit price is essentially a function of time-dependent variables. Furthermore, since servers are allowed to differentiate the unit price of each service task for profit maximization [30], we drop the explicit dependency on time-varying factors and represent the unit price as a general service-specific time-varying variable $p_s(t)$, to provide a more flexible and scalable pricing function. This representation allows providers in different MEC scenarios to integrate their own pricing models into our system, simply by specifying the current time slot and service type.

Considering that delayed tasks have a negative impact on the revenue, we use a more realistic metric to represent the revenue obtained by server i at slot t as:

$$g_i(t) = \sum_{s \in \mathcal{S}} \mu_{i,s}(t) p_s(t), \tag{4}$$

where $\mu_{i,s}(t)$ is the total amount of computation resources allocated to service tasks s that were *processed within delay budgets* by server i at slot t.

In the context of diverse real-world MEC scenarios, the influence of service performance and operation costs on service providers' profits may *vary*. For example, in remote operation scenarios, service providers would prioritize achieving reliable and stable user experience, even at the expense of higher operation costs. Therefore, a profit weight factor Δ is introduced to *customize* the impact of service performance and operation costs on the profit for service providers. The weighted profit of edge server i at slot t can be given by:

$$f_i(t) = \Delta g_i(t) - (c_i^1(t) + c_i^2(t) + c_i^3(t)). \tag{5}$$

In experiment VI-E, we further discuss the effectiveness of EdgeTimer under different profit weight factors.

E. Problem Formulation

Our objective is to *maximize the long-term profit* by jointly optimizing timescales that are used to update three-layer scheduling decisions. The control variables of three scheduling layers are $a_i^1(t)$, $a_i^2(t)$, and $a_i^3(t)$, indicating *whether current placement, offloading, and allocation decisions of edge server i need to be updated, respectively*. Existing scheduling algorithms act as built-in rules to provide candidate scheduling decisions for placement $\widetilde{x}_{i,s}(t)$, offloading $\widetilde{y}_{i,j,s}(t)$, and allocation $\widetilde{z}_{i,s}(t)$. Given a task s, its current scheduling decision performed at server i depends on the current control policy of server i. We take the service placement process to exemplify this relationship.

At time slot t, the placement decision of edge server i for task s is denoted as $x_{i,s}(t)$. If the control policy of the service placement decides to update the current placement decision at server i, i.e., $a_i^1(t) = 1$, the server i would perform placement decision $\widetilde{x}_{i,s}(t)$ generated by existing placement rules. Conversely, the server i would continue the previous decision, i.e., $x_{i,s}(t-1)$. Thus, the placement decision performed at server i at slot t can be expressed as:

$$x_{i,s}(t) = a_i^1(t)\widetilde{x}_{i,s}(t) - (a_i^1(t) - 1)x_{i,s}(t-1). \tag{6}$$

The mapping between scheduling decisions and control policies at Layer-2 and Layer-3 scheduling can be defined similarly to (6).

Then, the joint optimization problem can be formulated as:

$$\widetilde{\mathcal{P}}\text{: } \max \mathbb{E}\left[\sum_{t=0}^{T}\sum_{i \in \mathcal{N}}(\Delta g_i(t) - a_i^1(t)c_i^1(t) - a_i^2(t)c_i^2(t) - a_i^3(t)c_i^3(t))\right]$$
$$\text{s.t.} \quad a_i^1(t), a_i^2(t), a_i^3(t) \in \{0,1\}, \forall t, i, \tag{7}$$

where T is the length of the observation period that can be adjusted based on the provider's practical needs.

IV. EdgeTimer Design

In EdgeTimer, we design a three-layer hierarchical DRL framework to decouple the joint optimization problem. Then, the multi-agent DRL algorithm for each sub-problem is explained. Furthermore, we develop a system defender to ensure policy reliability.

A. Three-Layer Hierarchical DRL for Problem Decomposition

Achieving the objective in (7) relies on performing three layers of tightly coupled decision-making. The higher-layer decisions determine lower-layer feasible regions, and lower-layer scheduling results reflect the effectiveness of higher-layer decisions. This makes it intractable to generate three effective decisions simultaneously. For ease of learning, we decouple the original problem into three independent sub-problems, and design a three-layer hierarchical deep reinforcement learning (HDRL) framework to solve different sub-problems in a coordinated manner.

We decouple the original problem ($\widetilde{\mathcal{P}}$) into sub-problems ($\mathcal{P}1, \mathcal{P}2, \mathcal{P}3$) by decomposing the total objective into respective sub-objective of different layers of scheduling. Each sub-objective is determined by the current decision at that layer and the decisions from the previous layer.

Edge-cloud service placement sub-problem ($\mathcal{P}1$): The placement cost is derived from (1). Although the revenue cannot be directly decomposed, the extent of service coverages can potentially measure the contribution of current placement decisions to the total revenue, as verified in Section VI-F. The service placement sub-problem can be formulated as:

$$\mathcal{P}1: \max \mathbb{E}\left[\sum_{t=0}^{T}\alpha d(t) - \sum_{t=0}^{T}\sum_{i \in \mathcal{N}} a_i^1(t)c_i^1(t)\right]$$
$$\text{s.t.} \quad a_i^1(t) \in \{0,1\}, \forall t, i, \tag{8}$$

where $d(t)$ is the amount of computation demands of "covered" tasks after performing service placement decision at slot t and resource allocation decision at slot $t-1$. A task is considered "covered" if the edge server serving it has already cached the services required to process this task. Please note that

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on June 30,2026 at 07:30:06 UTC from IEEE Xplore. Restrictions apply.

the three-layer scheduling architecture performs following a sequential loop: Layer-1, Layer-2, and Layer-3 scheduling decisions. Therefore, for the sub-problem corresponding to Layer-1 scheduling, the decision from the previous layer is Layer-3's decision from the previous time slot, i.e., the resource allocation decision of the sub-problem. The coefficient α is to weigh the impact of service coverages.

Edge-edge task offloading sub-problem ($\mathcal{P}2$): Given the offloading sub-cost in (2), a straightforward option to formulate the offloading sub-problem is to set the transmission delay budget and maximize the difference between total computation demands of current tasks, $u(t)$, that complete offloading within the transmission delay budget and the sub-cost. The variable $u(t)$ is jointly determined after the current decisions of service placement and task offloading. However, the above option is not equivalent to maximizing the original profit, as it may result in offloading too many workloads to the nearest edges, inducing high computation delay and revenue loss. To avoid load imbalance, we introduce a load balancing function $v(\cdot)$ that maps current placement and offloading decisions to the workload variance of all edge servers. The task offloading sub-problem can be given by:

$$\mathcal{P}2: \max \mathbb{E}\left[\sum_{t=0}^{T}\beta u(t) - \sum_{t=0}^{T}\sigma v(t) - \sum_{t=0}^{T}\sum_{i\in\mathcal{N}} a_i^2(t)c_i^2(t)\right]$$
$$\text{s.t.} \quad a_i^2(t) \in \{0,1\}, \forall t, i, \tag{9}$$

where β and σ are coefficients to control the degree of the offloaded task satisfaction and workload variance, respectively.

Intra-edge resource allocation sub-problem ($\mathcal{P}3$). Similarly, we set computation delay budgets for service requests, and denote the total computation demands of current tasks processed within computation delay budgets and the corresponding weight coefficient as $l(t)$ and ϕ, respectively. The resource allocation sub-problem is defined by:

$$\mathcal{P}3: \max \mathbb{E}\left[\sum_{t=0}^{T}\phi l(t) - \sum_{t=0}^{T}\sum_{i\in\mathcal{N}} a_i^3(t)c_i^3(t)\right]$$
$$\text{s.t.} \quad a_i^3(t) \in \{0,1\}, \forall t, i. \tag{10}$$

The variable $l(t)$ in (10) is determined after performing the decisions of the current decisions of task offloading and resource allocation.

Workflow of three-layer hierarchical DRL framework: Hierarchical deep reinforcement learning (HDRL) [46] is a promising framework to extend traditional DRL algorithms to learn complicated tasks, by breaking the overall decision-making down into a hierarchy of sub-tasks. As shown in Fig. 3, we design a tailored HDRL framework that comprises three-layer DRL controllers, i.e., ⓐ Layer-1 controller, ⓑ Layer-2 controller, and ⓒ Layer-3 controller, to cope with three layers of scheduling sub-problems $\mathcal{P}1$, $\mathcal{P}2$, and $\mathcal{P}3$, respectively. Each DRL controller consists of multiple DRL agents. The detailed settings of multiple DRL agents are described in Section IV-B.

At slot t, the input of each layer of the controllers is the layer-specific state composed of two components, i.e., the status

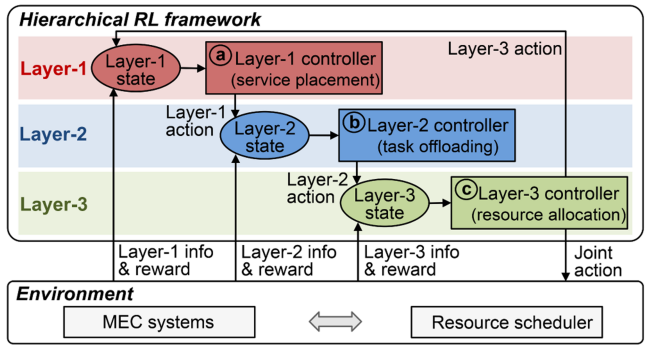


Fig. 3. The workflow of three-layer hierarchical DRL framework.

of queueing tasks after executing the action of the layer above it and the current resource configuration information from the environment. The DRL controller uses the state to select the action that determines whether to update the current scheduling decisions of the corresponding layer.

After performing the joint three-layer action, the environment transitions to slot $t+1$ with new three-layer scheduling information, and provides three-layer controllers with respective rewards. The reward for each controller is set based on the targeted sub-problem's objective. The controllers use these rewards to improve their actions for future time slots. The detailed settings of three-layer states, actions, rewards and DRL training are described in Section IV-B.

B. Multi-Agent DRL for Decentralized Scheduling

Traditional centralized scheduling leads to significant communication costs due to frequent information exchanges between central cloud servers and edge servers [15]. To save the communication cost, *each edge server is required to use its local information to make decisions independently without relying on the global information from other servers.* To achieve this goal, each DRL controller uses the tailored multi-agent DRL algorithm to generate decentralized actions.

Decentralized POMDP modeling: We formulate the above learning process as a decentralized partially observable Markov decision process (Dec-POMDP) [47]. For each of the controllers, *each edge server acts as an agent* to learn decentralized actions. In addition to conventional components of a Markov decision process (MDP), such as the state space $\mathcal{S}$, the action space $\mathcal{A}$, the state transition function $p: \mathcal{S}\times\mathcal{A}\to\mathcal{S}$, the shared reward function $r: \mathcal{S}\times\mathcal{A}\to\mathbb{R}$, and the discount factor γ, a set of extra local observations $\mathcal{O} = \times_{i\in\mathcal{N}} o_i$ is considered in Dec-POMDP to reflect local observations available to each agent from the global state $\mathcal{S}$.

Each agent i uses its policy $\pi_i(a_i(t)|o_i(t))$ to select the action $a_i(t)$ from the partial observations $o_i(t)$. The goal of the Dec-POMDP is to find a joint policy of all agents $\boldsymbol{\pi} = \{\pi_i(a_i(t)|o_i(t))|i\in\mathcal{N}\}$ that maximizes the expected cumulative reward:

$$\max_{\boldsymbol{\pi}} \mathbb{E}\left[\sum_{t=0}^{\infty}\gamma^t \boldsymbol{r}(\boldsymbol{s}(t), \boldsymbol{\pi}(\boldsymbol{a}(t)|\boldsymbol{o}(t)))\right], \tag{11}$$

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on June 30,2026 at 07:30:06 UTC from IEEE Xplore. Restrictions apply.

TABLE II
SETTINGS OF DRL COMPONENTS FOR EACH LAYER OF THE CONTROLLERS CONSIDERED

Layers of DRL controllers	States of each agent	Actions of each agent	Rewards for all agents
Layer-1 controller edge-cloud service placement	extent of service coverages, available memory, installed service types, service types of arrived tasks	whether to update the current placement decisions of its own agent	$\sum_{i\in\mathcal{N}}[(a\cdot \text{PlaceCost}_i +b\cdot(\text{TaskUnserved}_i-\text{DudgetUnserved}))^{-1}]$
Layer-2 controller edge-edge task offloading	ratio of tasks that violate delay budgets, available CPU unit, amount of unofloaded workloads	whether to update the current offloading decisions of its own agent	$\sum_{i\in\mathcal{N}}[(c\cdot\text{OffloadCost}_i+d\cdot(\text{TaskDelay}_i -\text{DudgetDelay}))^{-1}]+\text{WorkloadVariance}$
Layer-3 controller intra-edge resource allocation	ratio of tasks that violate delay budgets, rate variance among tasks, available CPU unit, workloads waiting for resource allocation	whether to update the current allocation decisions of its own agent	$\sum_{i\in\mathcal{N}}[\text{UndelayedWorkload}_i/(e\cdot\text{AllocateCost}_i +f\cdot(\text{TaskDelay}_i-\text{DudgetDelay}))]$

Algorithm 1: EdgeTimer for Adaptive Multi-Timescale Resource Scheduling.

/* **Initialization** */
Initialize parameters of neural networks, actions $(a_i^1(0),a_i^2(0),a_i^3(0))$, and replay buffers (R^1,R^2,R^3);
Initialize the network configuration $\widetilde{s}$;
for epoch $=0,\cdots,M$ **do**
  for time slot $t=1,\cdots,T$ **do**
    for each edge server $i=1,\cdots,N$ **do**
      /* **Generate Layer-1 actions** */
      $o_i^1(t)\leftarrow \text{STEP}(a_i^1(t-1),a_i^3(t-1))$;
      $a_i^1(t)\leftarrow \pi_i^1(o_i^1(t)|\theta_i^1)$;
      /* **Generate Layer-2 actions** */
      $o_i^2(t)\leftarrow \text{STEP}(a_i^2(t-1),a_i^1(t))$;
      $a_i^2(t)\leftarrow \pi_i^2(o_i^2(t)|\theta_i^2)$;
      /* **Generate Layer-3 actions** */
      $o_i^3(t)\leftarrow \text{STEP}(a_i^3(t-1),a_i^2(t))$;
      $a_i^3(t)\leftarrow \pi_i^3(o_i^3(t)|\theta_i^3)$;
    /* **Compute three-layer rewards** */
    Perceive rewards $r_{i\in\mathcal{N}}^1(t)$, $r_{i\in\mathcal{N}}^2(t)$, and $r_{i\in\mathcal{N}}^3(t)$;
    Store transitions of three controllers in R^1, R^2, and R^3, respectively;
    if $\text{DONE}(o_{i\in\mathcal{N}}^1(t),o_{i\in\mathcal{N}}^2(t),o_{i\in\mathcal{N}}^3(t))$ **then**
      $o_{i\in\mathcal{N}}^1(t),o_{i\in\mathcal{N}}^2(t),o_{i\in\mathcal{N}}^3(t)\leftarrow \text{RESET}(\widetilde{s})$;
  Sample mini-batch W^1, W^2, and W^3 from R^1, R^2, and R^3 as training samples, respectively;
  Update parameters of networks using (12)–(14);

where $\boldsymbol{s}(t)$, $\boldsymbol{a}(t)$, and $\boldsymbol{o}(t)$ are global states, actions, and observations of all agents at slot t, respectively. The variable $\boldsymbol{r}(t)$ is the shared reward of all DRL agents.

The layer-specific state, action, and reward are summarized in Table II. In experiment VI-D, we investigate the impact of parameters of the reward function on the overall profit performance.

We design a multi-agent DRL (MADRL) to achieve the goal of Dec-POMDP in (11). As shown in Fig. 4, we adopt a *centralized training and decentralized execution* scheme. Specifically, each agent has two neural networks, where an actor network learns to generate the policy with local observations during the online phase and a critic network approximates the centralized value function from the global environment to evaluate the current policy during the offline phase. The training process is shown in Algorithm 1.

During online inference, the main challenge of decentralized scheduling is enabling each DRL agent to make decisions that

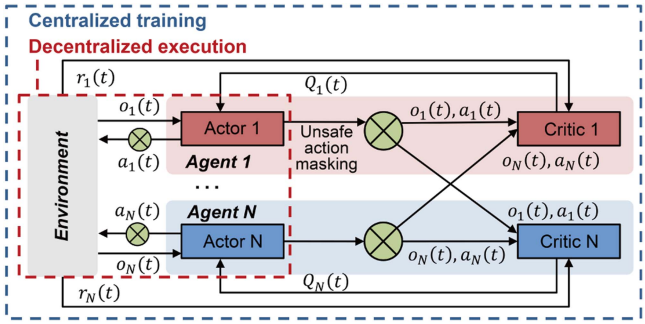


Fig. 4. The illustration of multi-agent DRL algorithm.

effectively collaborate with other agents to maximize overall profit, using partial observations. To compensate for the information loss, during the offline training phase (before real-world deployment), we allow the critic networks to use global information from other agents to evaluate and improve decentralized policies. This centralized training scheme achieves effective collaboration among agents, by sharing inter-agent information and implementing shared joint rewards.

Decentralized execution with actors: For each agent $i\in\mathcal{N}$, its actor i maps a local observation $o_i(t)$ to an action $a_i(t)$ with policy π_{θ_i} parameterized by θ_i. To maximize the return obtained by the policy in (11), the actor of agent i is updated following the policy gradient theorem [48]:

$$\nabla_{\theta_i}J(\pi_{\theta_i})=\mathbb{E}_{\pi_{\theta_i}}[\nabla_{\theta_i}\log\pi_{\theta_i}(a_i(t)|o_i(t))Q_i(\boldsymbol{o}(t),\boldsymbol{a}(t))], \quad (12)$$

where $Q_i(\boldsymbol{o}(t),\boldsymbol{a}(t))$ is the centralized action-value function that receives global observations, $\boldsymbol{o}(t)=(o_1(t),\ldots,o_N(t))$, together with actions of all agents, $\boldsymbol{a}(t)=(a_1(t),..,a_N(t))$, and outputs a measure of policy performance for agent i. The action-value function $Q_i(\boldsymbol{o}(t),\boldsymbol{a}(t))$ indicates the expected return of selecting action $\boldsymbol{a}(t)$ in observations $\boldsymbol{o}(t)$ under policy π_{θ_i}. We design an individual critic network for agent i to approximate $Q_i(\boldsymbol{o}(t),\boldsymbol{a}(t))$.

Centralized training with critics: The critic Q_{ω_i} of agent i with parameter ω_i can be trained by minimizing the mean-square error as follows:

$$L(\omega_i)=(Q_{\omega_i}(\boldsymbol{o}(t),\boldsymbol{a}(t))-\overline{Q}_i(t))^2, \quad (13)$$

$$\overline{Q}_i(t)=r_i(t)+\gamma Q_{\omega_i}(\boldsymbol{o}(t+1),\boldsymbol{\pi}(\boldsymbol{o}(t+1))). \quad (14)$$

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on June 30,2026 at 07:30:06 UTC from IEEE Xplore. Restrictions apply.

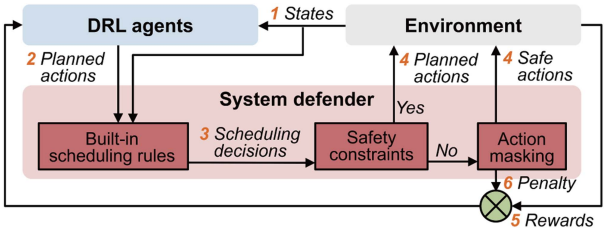


Fig. 5. The illustration of system defender with safe learning mechanism.

C. System Defender for Safe Learning

DRL-based algorithms excel in dynamic decision-making but may generate unsafe actions due to the uncertainty of neural networks. Unsafe actions, a concept from safe DRL [49], [50], refer to decisions made by DRL agents that could cause harm. Algorithm engineers typically define unsafe actions based on the specific problem, with the exact definition varying according to the system's objectives and the users' requirements [51], [52]. In our problem, unsafe actions are refined into the following three types.

Unconstrained actions: In decentralized multi-agent DRL environment, agents may generate unconstrained actions that do not satisfy resource limits. For example, at slot t, multiple agents (edge servers) offload their respective task requests to the same destination server. However, due to the lack of communication among agents, this scheduling process may result in the total computation workload on the destination server exceeding its computation capacity, leading to high computation delay.

Short-sighted actions: The short-sighted actions ignore the executed actions of other scheduling layers, resulting in inter-layer action conflicts. For example, we assume that for edge i at slot $t-1$, its Layer-2 scheduling decision was to offload task s to edge j. At slot t, if the Layer-1 placement controller removes service s from edge j, then the edge i's Layer-2 action of continuing previous offloading decisions ($a_i^2(t)=0$) is referred to as a "short-sighted action". This is because that this action leads to task s not being offloaded, given that the Layer-1 controller has removed the necessary services from the destination server. Consequently, this incurs extra scheduling delay for these unoffloaded tasks.

Invalid actions: They are actions that do not impact the environment but lead to sparse reward issues, where the agents rarely receive useful reward signals. For example, the time interval of task arrivals becomes larger at night (e.g., hundreds of time slots), where no requests need to be scheduled for a long time. If we rely on DRL algorithms to select actions during this idle period, then the agents would receive a series of invalid rewards irrespective of action types, which harms the learning efficiency of DRL algorithms.

System defender design: Following the methodology in existing practices [51], [52], we define a system as reliable if it does not take unsafe actions. To ensure system reliability in production, we design a system defender with a safe learning mechanism, as shown in Fig. 5. The system defender disrupts the direct interaction between the system environment and the agents. Actions must undergo safety examination and repairing before being applied to the environment.

Specifically, the system defender first takes planned actions from DRL agents and network states as inputs, and generates current scheduling decisions using built-in scheduling rules (e.g., Kubernetes rules). Then, we build a set of safety constraints to check whether the planned actions are safe. Note that unlike some safe DRL scenarios [53] that rely on the next slot's state to identify unsafe actions, in our problem, all three types of unsafe actions can be deterministically identified by comparing the scheduling decisions with current system states (e.g., check whether the destination server has installed the corresponding services).

If the safety constraints are satisfied, the planned actions are directly applied to the environment. Otherwise, we use the action masking mechanism to repair the planned actions. We observe that the action of updating the current scheduling decision is neither unconstrained nor shortsighted. This is because when we decide to update the current scheduling decision, built-in rules (e.g., Kubernetes scheduling rules) are used to determine a new scheduling decision. These built-in rules first determine the feasible domain based on servers' available resources and pending task demands. Servers that do not satisfy safety constraints are excluded from the feasible domain. Based on this observation, for unconstrained or shortsighted actions, we directly replace them with actions of updating the current scheduling decisions to ensure safety. For invalid actions, since there are no task demands at corresponding time slots, we simply skip those idle slots until tasks arrive. Trajectories associated with invalid actions will not be stored in the replay buffer to avoid sparse rewards. This approach allows us to deterministically replace all unsafe actions with safe ones, thereby guaranteeing system reliability.

Finally, to prevent agents from choosing unsafe actions in the future, we impose an additional penalty, i.e., negative rewards, on the rewards obtained from the environment.

D. Convergence and Complexity Analysis

In this subsection, we theoretically analyze the convergence and computational complexity of EdgeTimer.

Convergence analysis: As discussed in Section IV-A, EdgeTimer uses three-layer DRL controllers to coordinate heterogeneous scheduling layers. Each layer of the controllers adopts multi-agent DRL for learning a joint policy of all agents $\boldsymbol{\pi}=\{\pi_i|i\in\mathcal{N}\}$ to maximize the expected cumulative reward. We use the action-value function to evaluate the effectiveness of policy π_i as follows:

$$\begin{aligned} Q_i(\boldsymbol{o}(t),\boldsymbol{a}(t)) &= \mathbb{E}\left[\sum_{k=0}^{\infty}\gamma^k \boldsymbol{r}(t+k)\mid \boldsymbol{o}(t),\boldsymbol{a}(t)\right] \\ &= \mathbb{E}_{\boldsymbol{a}\sim\boldsymbol{\pi}}\left[\sum_{k=0}^{\infty}\gamma^k \boldsymbol{r}(\boldsymbol{s}(t+k),\boldsymbol{a}(t+k))\mid \boldsymbol{o}(t),\boldsymbol{a}(t)\right] \\ &= \mathbb{E}_{\substack{a_i\sim\pi_i \\ a_{-i}\sim\pi_{-i}}}\left[\sum_{k=0}^{\infty}\gamma^k \boldsymbol{r}(\boldsymbol{s}(t+k),a_i(t+k),a_{-i}(t+k))\,|\boldsymbol{o}(t),\boldsymbol{a}(t)\right], \end{aligned} \tag{15}$$

where $-i$ denotes the indices of all agents except for agent i.

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on June 30,2026 at 07:30:06 UTC from IEEE Xplore. Restrictions apply.

We can observe that at each controller layer, an agent's performance is determined not only by its own decisions but also by the behaviors of other agents at the same layer. Furthermore, due to inter-layer information communication, the performance of an agent is further affected by the choices made by agents at different controller layers. For convenience, we investigate the convergence in a multi-agent environment by *unifying* all the above influencing factors to the *agent level*. The action-value function of agent i can be denoted by $Q_i^{\pi_i,\pi_{-i}}(\boldsymbol{o}(t),\boldsymbol{a}(t))$.

In order to prove the convergence, we begin by introducing the concept of Nash equilibrium [54] to define the optimal policy in a multi-agent environment.

Definition 1 (Nash Equilibrium): The joint policy $\boldsymbol{\pi}^* = \{\pi_1^*,\ldots,\pi_N^*\}$ is a Nash equilibrium when each individual policy π_i^* is the best response to other policies π_{-i}^*. That is, for $\forall i \in \mathcal{N}$, $\forall \boldsymbol{o}(t) \in \mathcal{O}$, and $\forall \pi_i$

$$Q_i^{\pi_i^*,\pi_{-i}^*}(\boldsymbol{o}(t),\boldsymbol{a}(t)) \geq Q_i^{\pi_i,\pi_{-i}^*}(\boldsymbol{o}(t),\boldsymbol{a}(t)). \tag{16}$$

The action-value received in the Nash equilibrium is called the Nash Q-value [55], i.e., $Q^{\boldsymbol{\pi}^*}(\boldsymbol{o}(t),\boldsymbol{a}(t)) = \{Q_1^{\pi_1^*,\pi_{-1}^*}(\boldsymbol{o}(t),\boldsymbol{a}(t)),\ldots,Q_N^{\pi_N^*,\pi_{-N}^*}(\boldsymbol{o}(t),\boldsymbol{a}(t))\}$. We now demonstrate the convergence of our algorithm's action-value function to the Nash Q-value over time slots. To this end, we first make three assumptions as follows.

Assumption 1: For each time slot, the action-value function of each agent can be represented using a set of linearly independent functions.

Assumption 2: Every state-action pair is visited infinitely often.

Assumption 3: For all t, $\boldsymbol{o}(t) \in \mathcal{O}$, $i \in \mathcal{N}$, the Nash equilibrium $\boldsymbol{\pi}^* = \{\pi_1^*,\ldots,\pi_N^*\}$ is identified as either a saddle point or the global optimum point.

Then, the convergence of EdgeTimer can be stated as follows:

Theorem 1 (Value-Function Convergence): If Assumptions 1, 2, and 3 are satisfied, then the action-value function $Q^{\boldsymbol{\pi}}(t)$ converges to the Nash Q-value $Q^{\boldsymbol{\pi}^*}$ w.p.1.

Proof: Refer to proof in [56].

If the above assumptions do not hold, the inherent uncertainty of neural networks makes it difficult to guarantee that our algorithm will converge to a Nash equilibrium. Alternatively, practice [57] with a similar multi-agent actor-critic DRL framework has proved that if enough samples are collected during importance sampling, the action-value function will ideally increase (policy improvement).

Complexity analysis: The computational complexity of EdgeTimer comes from the training and inference process, which heavily depends on multiplication operations within the deep neural networks. EdgeTimer has three layers of DRL controllers. Each DRL controller has N agents, each of which equips two neural networks, i.e., the actor and critic network. We assume that all actors and critics share the same fully-connected network structure with L layers. We denote the number of neurons in the lth layer of the actor and critic by N_l^a and N_l^c, respectively. For kth DRL controller, the input dimensions of the actor and critic are defined by I_k^a and I_k^c, respectively.

For the training process of kth DRL controller, its computational complexity of the actor network is $O(T_k^a) = O(I_k^a N_1^a + \sum_{l=1}^{L-1} N_l^a N_{l+1}^a + 2\tilde{} N_L^a)$. Similarly, the computational complexity of the critic is $O(T_k^c) = O(I_k^c N_1^c + \sum_{l=1}^{L-1} N_l^c N_{l+1}^c + N_L^c)$. The overall computational complexity is expressed as $O(HMN(\sum_{k=1}^{3}(T_k^a + T_k^c)))$, where H and M are the number of epochs and maximum steps per epoch, respectively.

During the inference process, only the actor networks are performed. The computational complexity for one-step execution is bounded by $O(N(\sum_{k=1}^{3}(T_k^a)))$.

V. Implementation

EdgeTimer was implemented on a realistic Vienna system-level simulator [18], embedding 45 combinations of typical scheduling rules for three-layer scheduling.

A. MEC System Simulator

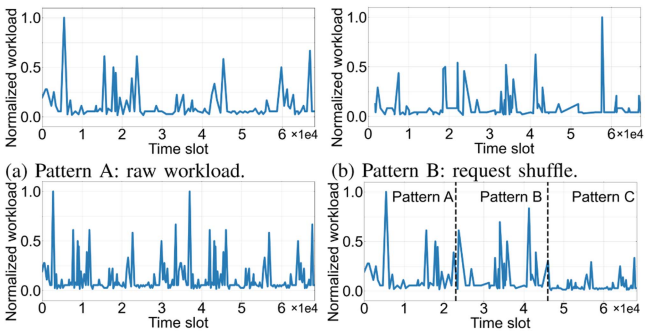
We implemented 12 edge servers and one cloud server in a realistic Vienna system-level simulator[2] [18] with MATLAB. In the simulation, edge servers were distributed over a 5 km$\times$ 5 km region. Each edge server was equipped with four CPU cores and eight GB memory. The cloud had eight CPU cores and 16 GB memory. The length of each time slot was one second. The unit price of computation resource was derived from the pay-as-you-go pricing model at Microsoft Azure [45]. To simulate operation costs, we measured the service migration time [12], handover time [13], and server lead time [14] in service placement, task offloading, and resource allocation, respectively, and converted them to the cost based on the hourly price at Microsoft Azure. For simplicity, we dropped the dependency on the money and service types and used relative weights to set the unit price, placement cost, offloading cost, and unit allocation cost as 25, $0.3d_1$, $0.1d_2$, and 0.5, respectively. Parameters d_1 and d_2 are the transmission distance of service migration and task offloading, respectively.

B. Integration With Representative Scheduling Rules

The simulator embedded a total of 45 combinations of representative scheduling rules for service placement, task offloading, and resource allocation. These built-in rules were executed when EdgeTimer generated the "update decision" action (discussed in Section III-E). The service placement rules included a Kubernetes placement rule-horizontal pod autoscaling (HPA) [58], Top-K rule [6], and always migration rule (AM) [59]. For task offloading, we implemented 5 offloading rules of Kubernetes [19] (MRP, LRP, RLP, SSP, and RCRP). The allocation rules consisted of proportional fairness (PF) [60], round robin (RR) [61], and equal allocation (EA).

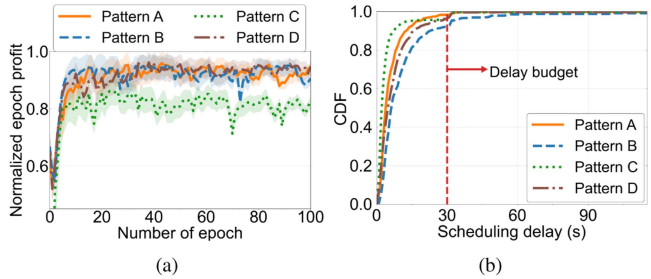
[2]The simulator enables to perform resource scheduling of 5G multi-tier networks. We extended it to MEC system scenarios by considering each base station as an edge server and adding an extra network node as a cloud.

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on June 30,2026 at 07:30:06 UTC from IEEE Xplore. Restrictions apply.



(a) Pattern A: raw workload. (b) Pattern B: request shuffle. (c) Pattern C: 2× request frequency. (d) Pattern D: A-B-C combination.

Fig. 6. Illustrations of four workload patterns.



(a) (b)

Fig. 7. (a) Normalized epoch profit of EdgeTimer and (b) CDF of the scheduling delay in EdgeTimer under different workload patterns.

C. Training Settings

The safe multi-agent DRL algorithm was implemented in Python 3.6.2 with PyTorch 1.10.2. Both the actor and critic networks included a one-layer RNN with 64 hidden units and were updated by the learning rate of 5×10^{-4}. The discount factor γ was set to 0.99, and parameter λ in GAE was 0.95.

VI. Evaluation

The evaluation aims to answer the following questions:

- How does EdgeTimer perform in different workload patterns compared with comparatives (Section VI-B)?
- How efficient is EdgeTimer with different built-in scheduling rules (Section VI-C)?
- What is the impact of EdgeTimer's key parameters on the profit performance (Sections VI-D and VI-E)?
- What is the contribution of each of our key designs in EdgeTimer (Section VI-F)?

A. Methodology

The training and inference were conducted with an Intel(R) Core CPU (2.90 GHz 8-cores) and 512 GB RAM.

Multi-pattern task workloads: We evaluated EdgeTimer on real-world cluster traces from Alibaba [20] that provided task information per second within 8 days, including start time, end time, task type, required CPU, and required memory. We considered "task type" in traces as the type of services, and divided task requests into 12 services. The workload of each service task was estimated by multiplying the number of CPU cores it required by its actual processing time. Following the methodology in KaiS [15], the delay budget of each service task was set by proportionally scaling its actual processing time. To evaluate EdgeTimer in task arrivals with different patterns, we generated multi-pattern trace as shown in Fig. 6. The pattern A was the raw workload under stochastic arrivals (Fig. 6(a)). In pattern B, we shuffled the order in which the edge server received tasks to reflect interference from other tasks (Fig. 6(b)). Furthermore, we doubled the request frequency in pattern C to simulate an increase in workloads (Fig. 6(c)), and concatenated the above three patterns as pattern D to evaluate the adaptability to pattern changes (Fig. 6(d)).

Comparatives: We compared EdgeTimer with four comparison algorithms: *Static single-timescale (SST)* [4], [11] that updates three-layer scheduling decisions at each slot; *Static multi-timescale (SMT)* [6], [15] that updates decisions of each layer via a fixed and specific timescale[3]; *Delay-triggered update (DT)* and *workload-triggered update (WT)* that update an edge's decisions when the delay and waiting workload in the edge exceed corresponding thresholds, respectively.

B. Performance on Multi-Pattern Workloads

We compared EdgeTimer with comparatives in four workload patterns (Fig. 6). The built-in scheduling rule used in this subsection is AM-MRP-EA, i.e., AM for service placement, MRP for task offloading, and EA for resource allocation.

Profit performance: Fig. 7(a) shows the normalized epoch profit of EdgeTimer versus the number of offline training epochs in different workload patterns. The epoch profit is normalized against the profit obtained in pattern D. It shows that EdgeTimer enables to support different workload patterns and converges within 20 epochs. Even for pattern C and D where the learning task becomes more intractable due to the workload spike and the pattern fluctuating, nearly equivalent epochs are required for convergence compared with that in the raw trace. The converged profit is lowest in pattern C. This is because that as the request frequency increases, more operation costs are required to complete tasks within the delay budget.

Fig. 8 demonstrates the online profit performance for EdgeTimer and four comparatives. The profits in this figure are determined based on (5). For each pattern, all profits are normalized against that of EdgeTimer. EdgeTimer achieves the highest profit, irrespective of workload patterns. For pattern A, EdgeTimer stably improves the profit for $1.47\times$, $1.27\times$, $1.33\times$, and $1.28\times$ than SST, SMT, DT, and WT approaches, respectively. In particular, EdgeTimer shines in high-workload patterns (pattern C) and high-fluctuating (pattern D), where EdgeTimer achieves $5.56\times$ and $9.1\times$ higher profit than comparatives, respectively.

Fig. 9 investigates the underlying efforts behind EdgeTimer. It shows that the effectiveness of EdgeTimer comes from adaptively adjusting the update frequency of scheduling decisions based on the current task performance, compared with SST

[3]For each layer, we iterated through the timescale set, $\{1, 10, 50, 100\}$, to select the optimal as its fixed timescale.

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on June 30,2026 at 07:30:06 UTC from IEEE Xplore. Restrictions apply.

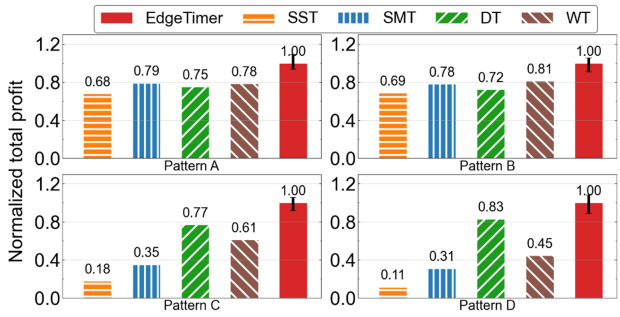


Fig. 8. Normalized total profit of EdgeTimer and comparatives in different workload patterns.

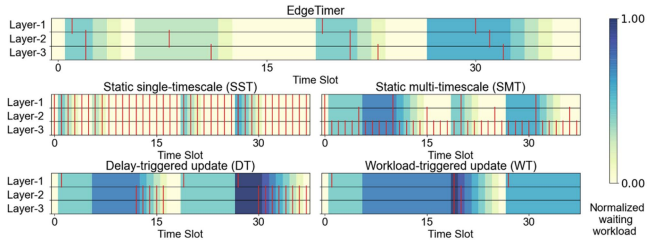


Fig. 9. Illustrations of updating timescales versus normalized waiting workloads for EdgeTimer and four comparatives. The scheduling decisions are updated at red vertical lines.

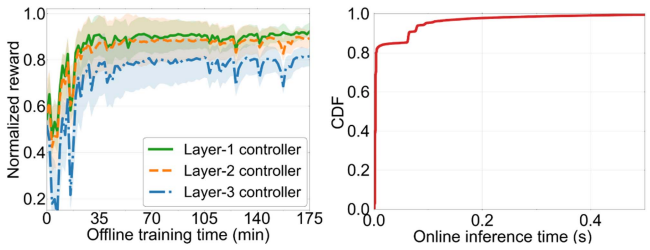


Fig. 10. (a) Normalized rewards of each layer of DRL controllers in EdgeTimer and (b) CDF of the online inference time in EdgeTimer.

and SMT. Furthermore, EdgeTimer outperforms DT and WT, as it enables to mitigate potential overload in advance using more comprehensive observations, rather than relying solely on the delay or workload metrics (cf. dense red vertical lines only appear at the end or beginning of the dark purple blocks).

Delay performance: Given a service type, Fig. 7(b) shows the CDF of the scheduling delay for such service requests under different workload patterns. We can observe that nearly all service requests can be completed within the delay budget, irrespective of workload patterns. It indicates that huge profits in EdgeTimer are not achieved at the expense of the user experience, i.e., delay performance.

Training and inference time: Fig. 10(a) demonstrates offline average rewards of Layer-1, Layer-2, and Layer-3 controllers in EdgeTimer. The overall controllers enable to achieve convergence within 35 minutes. Fig. 10(b) shows the CDF of the time spent by EdgeTimer to generate policies during the online process. The average inference time is 0.026 s which is less than the typical time interval between scheduling events, i.e., one second. These results indicate that EdgeTimer manages to execute scheduling policies at each time slot and imposes no extra overhead in production.

TABLE III
NORMALIZED PROFITS OF EDGETIMER UNDER DIFFERENT SETTINGS

Settings for EdgeTimer	Normalized Profits
State-of-the-art scheduling approach	0.79
Trained on pattern A, tested on pattern A	1.00 ± 0.07
Trained on pattern B, tested on pattern A	0.82 ± 0.15
Trained on pattern C, tested on pattern A	0.81 ± 0.15
Trained on pattern D, tested on pattern A	0.87 ± 0.12
Trained on pattern D with workload state, tested on pattern A	0.93 ± 0.08

Generalization: To evaluate the generalization ability of EdgeTimer, we trained EdgeTimer using different workload patterns and tested their profit performance on pattern A. There is no overlap between the test workload and training workload data. Table III shows the normalized profits obtained under various settings. All results are normalized using the profit achieved when EdgeTimer was trained on pattern A.

EdgeTimer performs well when it was trained on a workload that closely resembles the test workload (pattern A) or covers the similar distribution of the test workload (pattern D). When we trained EdgeTimer on pattern B or pattern C, it exhibits poor generalization. This is because EdgeTimer incorporates the learned distribution of the workload into its scheduling policy. If we add the future workload in the DRL state, EdgeTimer can further achieve 6.9% higher profits than training on raw pattern D. These results imply that an effective workload prediction can help EdgeTimer's policies be robust to workload variations. Furthermore, it emphasizes the effectiveness of using diverse training workloads to enhance the generalization.

C. Performance With Different Built-In Scheduling Rules

Table IV shows profit gains compared with existing approaches under 45 built-in scheduling rules in pattern A. EdgeTimer outperforms existing approaches for all built-in scheduling rules. For some Kubernetes rules, such as HPA-MRP-PF, EdgeTimer improves the profit performance for up to $2.43\times$ compared with approaches under the same rule.

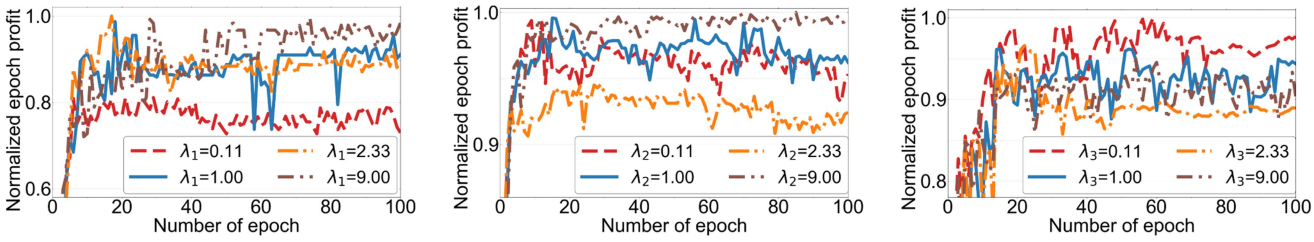
D. Impact of the Reward Weight Factor

Fig. 11(a)–(c) show the normalized epoch profit of EdgeTimer for different values of the three-layer reward weight factors, λ_1, λ_2 and λ_3, respectively. Parameters λ_1, λ_2, and λ_3 represent the ratios of weight factors in the reward functions for Layer-1, Layer-2, and Layer-3 controllers, respectively. Specifically, according to layer-specific rewards in Table II, three-layer reward weight factors are computed by: $\lambda_1 = a/b$, $\lambda_2 = c/d$ and $\lambda_3 = e/f$. The higher the value of λ_1, λ_2 and λ_3, the more the DRL agents focus on decreasing layer-specific operation costs. Conversely, the agents prioritize improving delay performance.

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on June 30,2026 at 07:30:06 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
PROFIT GAINS COMPARED TO EXISTING APPROACHES UNDER DIFFERENT BUILT-IN SCHEDULING RULES IN PATTERN A

		Service placement rule - resource allocation rule								
		HPA - PF	HPA - RR	HPA - EA	Top-K - PF	Top-K - RR	Top-K - EA	AM - PF	AM - RR	AM - EA
Task offloading rule	MRP	1.73-2.43x	1.70-2.39x	1.66-2.81x	1.40-3.87x	1.39-3.86x	1.40-3.55x	1.30-1.55x	1.31-1.56x	1.28-1.47x
	LRP	1.71-2.39x	1.70-2.39x	1.72-2.94x	1.34-3.85x	1.36-3.90x	1.53-3.56x	1.26-1.53x	1.25-1.52x	1.26-1.48x
	RLP	1.72-2.41x	1.71-2.40x	1.72-2.92x	1.31-3.83x	1.32-3.87x	1.48-3.60x	1.26-2.37x	1.27-2.38x	1.33-1.49x
	SSP	1.38-1.99x	1.39-2.01x	1.41-2.98x	1.91-4.67x	**1.92-4.70x**	1.29-3.61x	1.28-3.89x	1.26-3.83x	1.23-2.07x
	RCRP	1.72-2.41x	1.73-2.43x	1.70-2.92x	1.32-3.82x	1.30-3.78x	1.57-3.63x	1.26-1.59x	1.26-1.60x	1.30-1.50x



(a) Layer-1 reward weight factor. (b) Layer-2 reward weight factor. (c) Layer-3 reward weight factor.

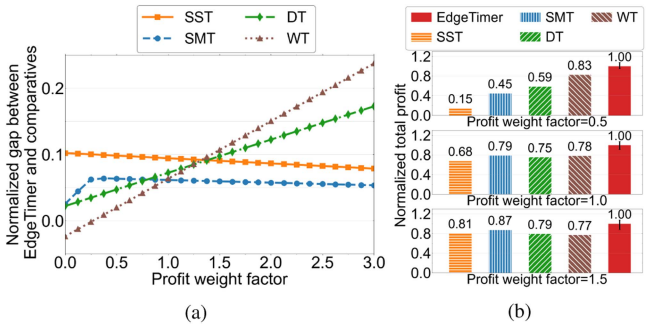
Fig. 11. Normalized epoch profit of EdgeTimer for different values of the three-layer reward weight factors, λ_1, λ_2, and λ_3, respectively.

In Fig. 11(a), we adopted SST to update scheduling decisions of Layer-2 and Layer-3, and evaluated the profit performance when $\lambda_1 = \{0.11, 1, 2.33, 9\}$. It shows that the convergence speed of the algorithm increases when $\lambda_1 = 0.11$. However, this improvement comes at the expense of a limited converged profit. The algorithm achieves its highest profit when λ_1 was set to 9. This is because for the Layer-1 service placement process, the demand for service reconfigurations is limited in the short period. Therefore, when $\lambda_1 = 9$, the agents manage to significantly improve the service providers' profits by prioritizing reducing the frequency of service configurations.

Fig. 11(b) demonstrates the algorithm's convergence for various values of λ_2, where we used SST for Layer-3 scheduling and set λ_1 to 9. Similarly, we observe that a larger λ_2 leads to higher overall profit, since a lower handover frequency can reduce operation costs without deteriorating service performance. After setting λ_2 to 9, we varied λ_3 in Fig. 11(c). Unlike the λ_1 and λ_2, a smaller value of λ_3 benefits the overall profit. This is because in pattern A, the requests in the resource pool undergo rapid variations over time slots. If the agents prioritize low-frequency updates in resource allocation decisions, it will adversely affect scheduling delay and subsequently impact profits. In real-world MEC systems, we suggest assigning larger reward weight factors to the Layer-1 and Layer-2 controllers, and smaller weight factors to the Layer-3 controller. The specific values should be determined based on the workload patterns.

E. Impact of the Profit Weight Factor

The above experiments were conducted when the profit weight factor (details in Section III-D), Δ, was set to 1. To investigate the impact of the profit weight factor, Fig. 12(a) shows the normalized gap between EdgeTimer and comparatives for different Δ. Compared with prevalent updating approaches,



(a) (b)

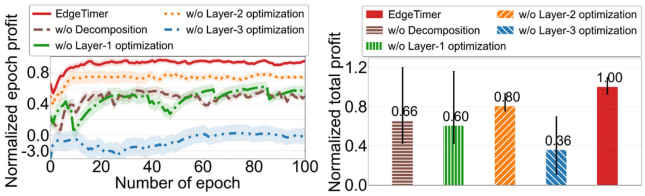
Fig. 12. (a) Normalized gap between EdgeTimer and comparatives for different profit weight factors and (b) normalized total profit of EdgeTimer and comparatives for different profit weight factors.

i.e., SST and SMT, EdgeTimer shines when Δ is small, where the profit improvement heavily relies on reducing operation costs. Fig. 12(b) shows that when $\Delta = 0.5$, EdgeTimer obtains $6.67\times$ and $2.22\times$ profit than SST and SMT, respectively. In contrast, the advantage of EdgeTimer over DT and WT grows as the value of Δ increases. This is because that DT and WT are driven by static thresholds that are initialized to large values. As Δ increases, DT and WT become incapable of automatically increasing the update frequency to ensure higher service quality.

F. Ablation Studies

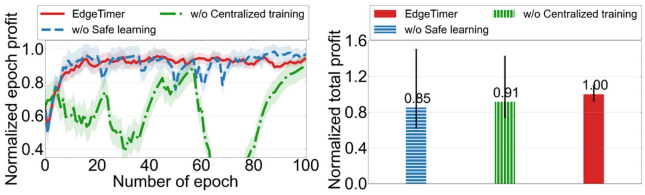
Impact of three-layer hierarchical DRL design: We design a three-layer hierarchical DRL framework to decompose the original task into three layers of sub-tasks and optimize them in a coordinated manner. To verify its contribution, Fig. 13 compared the offline and online profit of EdgeTimer, EdgeTimer without task decomposition, i.e., w/o Decomposition, and EdgeTimer without each layer of sub-task optimization in turn, i.e.,

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on June 30,2026 at 07:30:06 UTC from IEEE Xplore. Restrictions apply.



(a) Offline profits. (b) Online profits.

Fig. 13. Impact of the three-layer hierarchical DRL design on profit results.



(a) Offline profits. (b) Online profits.

Fig. 14. Impact of the safe multi-agent DRL design on profit results.

w/o Layer-1 optimization, w/o Layer-2 optimization, and w/o Layer-3 optimization. All results are normalized against that of EdgeTimer. EdgeTimer accelerates the convergence in the offline process and obtains $1.52\times$ higher online profit than w/o Decomposition, since the task decomposition scheme simplifies the learning task significantly. Moreover, removing any layer of optimization from EdgeTimer results in a decrease in profit of at least $1.25\times$, which reveals that each layer is indispensable for the overall performance.

Impact of safe multi-agent DRL design: Each task is learned by the safe multi-agent DRL algorithm, where centralized training and safe learning are two key components to learn decentralized decisions under partial observations and ensure system reliability, respectively. To verify their contributions, Fig. 14 compared the offline and online profit of EdgeTimer, EdgeTimer without centralized training, i.e., w/o Centralized training, and EdgeTimer without safe learning, i.e., w/o Safe learning. EdgeTimer manages to reach a stable higher profit than w/o Safe learning as it avoids high scheduling delay caused by unsafe actions. Furthermore, EdgeTimer overcomes the instability issues during the offline process encountered by w/o Centralized training, achieving $1.1\times$ higher online profit than that of w/o Centralized training. This is because centralized training can utilize comprehensive information to evaluate policies of actor networks, providing a more precise direction for network updates.

VII. Discussion

In this section, we discuss the system model, applicable scenarios, and limitations of EdgeTimer.

How to extend to other scheduling scenarios? We can extend our work beyond edge-cloud service placement, edge-edge task offloading, and intra-edge resource allocation through two simple modifications. We use an admission control scenario as an example. First, we need to embed existing scheduling rules for admission control [62] into EdgeTimer as built-in rules. When EdgeTimer decides to update admission control decisions, these rules determine which task requests to admit at each edge server. Second, we need to adjust the DRL-based controller by redefining its state, action, and reward. The state is required to include information about current queueing tasks and resource configuration, the action decides whether to update the admission control decisions, and the reward reflects both the operation cost of admission control and the scheduling delay introduced by these decisions.

How to extend to multi-provider scenarios? Our work supports two types of multi-provider scenarios: (i) Optimizing individual provider profits: EdgeTimer can maximize the individual profits of multiple providers by adjusting the built-in rules, states, and rewards in its DRL-based controller to align with each provider's scheduling policy, system configuration, and profit model. As shown in Section VI-B, EdgeTimer generalizes well across providers with similar configurations without retraining, and can adapt to significantly different configurations with minimal retraining (around 35 minutes). (ii) Collaborative profit maximization: Using our decentralized multi-agent DRL algorithm, EdgeTimer allows servers to independently make decisions while collaborating to maximize the total profit of all servers. The inter-server independence and shared objective enable our algorithm to optimize the overall profit of multiple servers managed by different providers, by treating each server as an independent DRL agent. However, in multi-provider scenarios, privacy concerns may limit inter-server information sharing, making decentralized scheduler training challenging. Addressing this issue lies beyond the scope of this paper and is left for future research.

How to incorporate extra practical factors affecting pricing? Given a service task type, establishing the relationship between time-varying influence factors and the unit price is equivalent to linking the unit price with time slots. Therefore, we model the unit price as a service-specific time-varying variable. This representation allows us to seamlessly incorporate new factors without affecting the model's effectiveness. For example, when providers consider the impact of available resources, we can derive the time-varying unit price for each service task based on the available resources at each time slot. Once the dynamic price is determined, we can update the price-oriented reward in the DRL controller, enabling it to adapt to new scenarios. The inherent nature of DRL algorithms ensures that our work can automatically learn policies through experience, without relying on predefined pricing models.

VIII. Conclusion

This paper investigates a new problem of maximizing service providers' profits by adaptively updating scheduling decisions in MEC. To address this problem, we propose EdgeTimer, an adaptive, asynchronous and autonomous multi-timescale scheduling framework based on deep reinforcement learning (DRL). EdgeTimer enables each edge server to independently update respective decisions for edge-cloud, edge-edge, and intra-edge scheduling in a coordinated manner. We develop three

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on June 30,2026 at 07:30:06 UTC from IEEE Xplore. Restrictions apply.

techniques in EdgeTimer. First, we design a tailored three-layer hierarchical DRL framework to facilitate the learning of coupled policies. Second, we integrate a multi-agent DRL algorithm to enable decentralized scheduling among edge servers. Third, a system defender component is introduced to guarantee system reliability. By conducting extensive experiments over real traces, EdgeTimer demonstrates a substantial improvement in profits across a wide range of workload patterns. Furthermore, EdgeTimer exhibits excellent scalability. It is not designed for specific scheduling rules, but rather offers a universal scheduling framework that can be extended to accommodate any heterogeneous scheduling rules.

REFERENCES

[1] Y. C. Hu, M. Patel, D. Sabella, N. Sprecher, and V. Young, "Mobile edge computing—A key technology towards 5G," *ETSI White Paper*, vol. 11, no. 11, pp. 1–16, 2015.

[2] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Commun. Surv. Tut.*, vol. 19, no. 4, pp. 2322–2358, Fourth Quarter 2017.

[3] F. Jiang, K. Wang, L. Dong, C. Pan, W. Xu, and K. Yang, "Deep-learning-based joint resource scheduling algorithms for hybrid MEC networks," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6252–6265, Jul. 2020.

[4] X. Ma, A. Zhou, S. Zhang, and S. Wang, "Cooperative service caching and workload scheduling in mobile edge computing," in *Proc. IEEE Conf. Comput. Commun.*, 2020, pp. 2076–2085.

[5] Q. Luo, S. Hu, C. Li, G. Li, and W. Shi, "Resource scheduling in edge computing: A survey," *IEEE Commun. Surv. Tut.*, vol. 23, no. 4, pp. 2131–2165, Fourth Quarter 2021.

[6] V. Farhadi et al., "Service placement and request scheduling for data-intensive applications in edge clouds," in *Proc. IEEE Conf. Comput. Commun.*, 2019, pp. 1279–1287.

[7] R. Han, S. Wen, C. H. Liu, Y. Yuan, G. Wang, and L. Y. Chen, "Edgetuner: Fast scheduling algorithm tuning for dynamic edge-cloud workloads and resources," in *Proc. IEEE Conf. Comput. Commun.*, 2022, pp. 880–889.

[8] X. Xiong, K. Zheng, L. Lei, and L. Hou, "Resource allocation based on deep reinforcement learning in IoT edge computing," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 6, pp. 1133–1146, Jun. 2020.

[9] X. Wang, J. Ye, and J. C. Lui, "Decentralized task offloading in edge computing: A multi-user multi-armed bandit approach," in *Proc. IEEE Conf. Comput. Commun.*, 2022, pp. 1199–1208.

[10] J. Zhang, S. Chen, X. Wang, and Y. Zhu, "DeepReserve: Dynamic edge server reservation for connected vehicles with deep reinforcement learning," in *Proc. IEEE Conf. Comput. Commun.*, 2021, pp. 1–10.

[11] K. Poularakis, J. Llorca, A. M. Tulino, I. Taylor, and L. Tassiulas, "Joint service placement and request routing in multi-cell mobile edge computing networks," in *Proc. IEEE Conf. Comput. Commun.*, 2019, pp. 10–18.

[12] F. Guo and M. Peng, "Efficient mobility management in mobile edge computing networks: Joint handover and service migration," *IEEE Internet Things J.*, vol. 10, no. 20, pp. 18237–18247, Oct. 2023.

[13] Y. Sun, S. Zhou, and J. Xu, "EMM: Energy-aware mobility management for mobile edge computing in ultra dense networks," *IEEE J. Sel. Areas Commun.*, vol. 35, no. 11, pp. 2637–2646, Nov. 2017.

[14] L. Jiao, A. M. Tulino, J. Llorca, Y. Jin, and A. Sala, "Smoothed online resource allocation in multi-tier distributed cloud networks," *IEEE/ACM Trans. Netw.*, vol. 25, no. 4, pp. 2556–2570, Aug. 2017.

[15] Y. Han, S. Shen, X. Wang, S. Wang, and V. C. Leung, "Tailored learning-based scheduling for kubernetes-oriented edge-cloud system," in *Proc. IEEE Conf. Comput. Commun.*, 2021, pp. 1–10.

[16] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[17] R. Lowe, Y. I. Wu, A. Tamar, J. Harb, O. PieterAbbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 6382–6393.

[18] M. K. Müller et al., "Flexible multi-node simulation of cellular mobile communications: The Vienna 5G system level simulator," *EURASIP J. Wireless Commun. Netw.*, vol. 2018, no. 1, pp. 1–17, 2018.

[19] Google kubernetes. Accessed: Feb. 20, 2023. [Online]. Available: https://kubernetes.io

[20] Alibaba cluster trace. Accessed: Feb. 20, 2023. [Online]. Available: https://github.com/alibaba/clusterdata

[21] Y. Hao, S. Yang, F. Li, Y. Zhang, S. Wang, and X. Ren, "EdgeTimer: Adaptive multi-timescale scheduling in mobile edge computing with deep reinforcement learning," in *Proc. IEEE Conf. Comput. Commun.*, 2024, pp. 671–680.

[22] T. Ouyang, R. Li, X. Chen, Z. Zhou, and X. Tang, "Adaptive user-managed service placement for mobile edge computing: An online learning approach," in *Proc. IEEE Conf. Comput. Commun.*, 2019, pp. 1468–1476.

[23] D. Bhatta and L. Mashayekhy, "A bifactor approximation algorithm for cloudlet placement in edge computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 8, pp. 1787–1798, Aug. 2022.

[24] Z. Zhou, Q. Wu, and X. Chen, "Online orchestration of cross-edge service function chaining for cost-efficient edge computing," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 8, pp. 1866–1880, Aug. 2019.

[25] T. Kim et al., "MoDEMS: Optimizing edge computing migrations for user mobility," in *Proc. IEEE/ACM 29th Int. Symp. Qual. Serv.*, 2022, pp. 1–2.

[26] L. Wang, L. Jiao, J. Li, J. Gedeon, and M. Mühlhäuser, "MOERA: Mobility-agnostic online resource allocation for edge computing," *IEEE Trans. Mobile Comput.*, vol. 18, no. 8, pp. 1843–1856, Aug. 2019.

[27] S.-H. Kim, S. Park, M. Chen, and C.-H. Youn, "An optimal pricing scheme for the energy-efficient mobile edge computation offloading with OFDMA," *IEEE Commun. Lett.*, vol. 22, no. 9, pp. 1922–1925, Sep. 2018.

[28] M. Liu and Y. Liu, "Price-based distributed offloading for mobile-edge computing with computation capacity constraints," *IEEE Wireless Commun. Lett.*, vol. 7, no. 3, pp. 420–423, Jun. 2018.

[29] J. Yan, S. Bi, L. Duan, and Y.-J. A. Zhang, "Pricing-driven service caching and task offloading in mobile edge computing," *IEEE Trans. Wireless Commun.*, vol. 20, no. 7, pp. 4495–4512, Jul. 2021.

[30] B. Baek, J. Lee, Y. Peng, and S. Park, "Three dynamic pricing schemes for resource allocation of edge computing for IoT environment," *IEEE Internet Things J.*, vol. 7, no. 5, pp. 4292–4303, May 2020.

[31] T. Bahreini, H. Badri, and D. Grosu, "Mechanisms for resource allocation and pricing in mobile edge computing systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 3, pp. 667–682, Mar. 2022.

[32] L. Ma, X. Wang, X. Wang, L. Wang, Y. Shi, and M. Huang, "TCDA: Truthful combinatorial double auctions for mobile edge computing in Industrial Internet of Things," *IEEE Trans. Mobile Comput.*, vol. 21, no. 11, pp. 4125–4138, Nov. 2022.

[33] X. Zheng, S. B. H. Shah, S. Usman, S. Mahfoudh, F. Shemim KS, and P. Kumar Shukla, "Resource allocation and network pricing based on double auction in mobile edge computing," *Springer J. Cloud Comput.*, vol. 12, no. 1, 2023, Art. no. 56.

[34] X. Wang, J. Ye, and J. C. S. Lui, "Mean field graph based D2D collaboration and offloading pricing in mobile edge computing," *IEEE/ACM Trans. Netw.*, vol. 32, no. 1, pp. 491–505, Feb. 2024.

[35] S. Cheng, T. Ren, H. Zhang, J. Huang, and J. Liu, "A Stackelberg-game-based framework for edge pricing and resource allocation in mobile edge computing," *IEEE Internet Things J.*, vol. 11, no. 11, pp. 20514–20530, Jun. 2024.

[36] Z. Liu, Y. Zhao, J. Song, C. Qiu, X. Chen, and X. Wang, "Learn to coordinate for computation offloading and resource allocation in edge computing: A rational-based distributed approach," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 5, pp. 3136–3151, Sep./Oct. 2022.

[37] C. Liu, Y. Zhong, R. Wu, S. Ren, S. Du, and B. Guo, "Deep reinforcement learning based 3D-trajectory design and task offloading in UAV-enabled MEC system," *IEEE Trans. Veh. Technol.*, vol. 74, no. 2, pp. 3185–3195, Feb. 2025.

[38] M. A. Hossain, W. Liu, and N. Ansari, "Computation-efficient offloading and power control for MEC in IoT networks by meta-reinforcement learning," *IEEE Internet Things J.*, vol. 11, no. 9, pp. 16722–16730, May 2024.

[39] H. Huang, Z.-Y. Chai, B.-S. Sun, H.-S. Kang, and Y.-J. Zhao, "Multiobjective deep reinforcement learning for computation offloading and trajectory control in UAV-base-station-assisted MEC," *IEEE Internet Things J.*, vol. 11, no. 19, pp. 31805–31821, Oct. 2024.

[40] Q. Liu, H. Zhang, X. Zhang, and D. Yuan, "Joint service caching, communication and computing resource allocation in collaborative MEC systems: A DRL-based two-timescale approach," *IEEE Trans. Wireless Commun.*, vol. 23, no. 10, pp. 15493–15506, Oct. 2024.

[41] Y. Gao et al., "Deep reinforcement learning-based trajectory optimization and resource allocation for secure UAV-enabled MEC networks," in *Proc. IEEE Conf. Comput. Commun. Workshops*, 2024, pp. 1–5.

[42] S. Chen, L. Wang, and F. Liu, "Optimal admission control mechanism design for time-sensitive services in edge computing," in *Proc. IEEE Conf. Comput. Commun.*, 2022, pp. 1169–1178.

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on June 30,2026 at 07:30:06 UTC from IEEE Xplore. Restrictions apply.

[43] X. Huang, B. Zhang, and C. Li, "Platform profit maximization on service provisioning in mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 70, no. 12, pp. 13364–13376, Dec. 2021.
[44] S. Zang, W. Bao, P. L. Yeoh, B. Vucetic, and Y. Li, "SOAR: Smart online aggregated reservation for mobile edge computing brokerage services," *IEEE Trans. Mobile Comput.*, vol. 22, no. 1, pp. 527–540, Jan. 2023.
[45] Pricing calculator of Microsoft azure. Accessed: Mar. 1, 2023. [Online]. Available: https://azure.microsoft.com/en-gb/pricing/calculator/
[46] T. D. Kulkarni, K. Narasimhan, A. Saeedi, and J. Tenenbaum, "Hierarchical deep reinforcement learning: Integrating temporal abstraction and intrinsic motivation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 3682–3690.
[47] F. A. Oliehoek, "Decentralized Pomdps," in *Reinforcement Learning: State-of-the-Art*. Berlin, Germany: Springer, 2012, pp. 471–503.
[48] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," in *Proc. Adv. Neural Inf. Process. Syst.*, 1999, pp. 1057–1063.
[49] A. Wachi and Y. Sui, "Safe reinforcement learning in constrained Markov decision processes," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 9797–9806.
[50] S. Gu et al., "A review of safe reinforcement learning: Methods, theories and applications," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 11216–11235, Dec. 2024.
[51] C. Liu, M. Xu, Y. Yang, and N. Geng, "DRL-OR: Deep reinforcement learning-based online routing for multi-type service requirements," in *Proc. IEEE Conf. Comput. Commun.*, 2021, pp. 1–10.
[52] H. Mao, M. Schwarzkopf, H. He, and M. Alizadeh, "Towards safe online reinforcement learning in computer systems," in *Proc. NeurIPS Mach. Learn. Syst. Workshop*, 2019.
[53] W. Saunders, G. Sastry, A. Stuhlmueller, and O. Evans, "Trial without error: Towards safe reinforcement learning via human intervention," 2017, *arXiv: 1707.05173*.
[54] K. Zhang, Z. Yang, and T. Başar, "Multi-agent reinforcement learning: A selective overview of theories and algorithms," 2021, *arXiv: 1911.10635*.
[55] M. L. Littman, "Value-function reinforcement learning in Markov games," *Elsevier Cogn. Syst. Res.*, vol. 2, no. 1, pp. 55–66, 2001.
[56] Z. Ning, Y. Yang, X. Wang, Q. Song, L. Guo, and A. Jamalipour, "Multi-agent deep reinforcement learning based UAV trajectory optimization for differentiated services," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 5818–5834, May 2024.
[57] Z. Wang, Z. Zhang, J. Wang, C. Jiang, W. Wei, and Y. Ren, "AUV-assisted node repair for IoUT relying on multi-agent reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 11, no. 3, pp. 4139–4151, Feb. 2024.
[58] Kubernetes placement rule-horizontal pod autoscaling. Accessed: May 1, 2023. [Online]. Available: https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale
[59] H. Ma, Z. Zhou, and X. Chen, "Leveraging the power of prediction: Predictive service placement for latency-sensitive mobile edge computing," *IEEE Trans. Wireless Commun.*, vol. 19, no. 10, pp. 6454–6468, Oct. 2020.
[60] F. Kelly, "Charging and rate control for elastic traffic," *Wiley Online Library Eur. Trans. Telecommun.*, vol. 8, no. 1, pp. 33–37, 1997.
[61] H. Aziz, X. Huang, N. Mattei, and E. Segal-Halevi, "The constrained round robin algorithm for fair and efficient allocation," 2019, *arXiv: 1908.00161*.
[62] R. Prasad and A. Sunny, "Scheduling slice requests in 5G networks," *IEEE/ACM Trans. Netw.*, vol. 31, no. 6, pp. 3025–3036, Dec. 2023.



Yijun Hao received the bachelor's degree from Northwestern Polytechnical University (NWPU), in 2019. She is currently working towards the PhD degree in the School of Mathematics and Statistics with Xi'an Jiaotong University. Her research interests include network optimization, reinforcement learning, and Edge-Cloud intelligence.



Shusen Yang (Senior Member, IEEE) received the PhD degree in computing from Imperial College London, in 2014. He is a professor and director of the National Engineering Laboratory for Big Data Analytics, and deputy director of Ministry of Education (MoE) Key Lab for Intelligent Networks and Network Security, both with Xi'an Jiaotong University (XJTU), Xi'an, China. He is also the deputy director of the Industrial Artificial Intelligence Center with Pazhou Laboratory, Guangzhou, China. Before joining XJTU, he worked as a lecturer (assistant professor) with the University of Liverpool from 2015 to 2016, and a research associate at Intel Collaborative Research Institute (ICRI) on sustainable connected cities from 2013 to 2014. Dr. Yang is a DAMO Academy Young Fellow, and an honorary research fellow with Imperial College London. He is a member of ACM. His research focuses on distributed systems and data sciences, and their applications in industrial scenarios, including data-driven network algorithms, distributed machine learning, Edge-Cloud intelligence, industrial internet, and industrial intelligence.



Fang Li received the bachelor's degree from Northeastern University (NEU), in 2021. She is currently working towards the PhD degree in the School of Mathematics and Statistics with Xi'an Jiaotong University. Her research interests include network optimization, reinforcement learning, and Edge-Cloud intelligence.



Yifan Zhang received the bachelor's degree from the University of Electronic Science and Technology of China, in 2022. He is currently working towards the MSc degree in the School of Mathematics and Statistics with Xi'an Jiaotong University. His research interests include network optimization and reinforcement learning.



Shibo Wang received the bachelor's degree from Xi'an Jiaotong University (XJTU), in 2019. He is currently working towards the PhD degree in the School of Mathematics and Statistics with Xi'an Jiaotong University. He is also currently working as a research assistant at the Department of Computer Science and Engineering with the Chinese University of Hong Kong (CUHK). His research interests include congestion control, real-time communication, 360-degree video streaming, and video analytics.



Xuebin Ren (Member, IEEE) received the PhD degree from the Department of Computer Science and Technology, Xi'an Jiaotong University, in 2017. He is currently an associate professor with Xi'an Jiaotong University and a member of the National Engineering Laboratory for Big Data Analytics (NEL-BDA). He has been a visiting PhD student with the Department of Computing, Imperial College London, from 2016 to 2017. His research interests include data privacy protection, federated learning, and privacy-preserving machine learning.

Authorized licensed use limited to: NORTHWESTERN POLYTECHNICAL UNIVERSITY. Downloaded on June 30,2026 at 07:30:06 UTC from IEEE Xplore. Restrictions apply.